FRANCO PIPITO

Backend / Full Stack Developer | Node.js · Java · Spring Boot · MySQL

Buenos Aires, Argentina | franpipito7@gmail.com | +54 11 2664 6239
LinkedIn: linkedin.com/in/francopipito | GitHub: github.com/franpipito | Portfolio: portfoliofranpipito.netlify.app

Perfil profesional

Desarrollador backend con ownership end-to-end de sistemas en producción. En TodoMicro soy responsable de las aplicaciones internas con las que opera la empresa: APIs REST en Node.js y Express sobre MySQL, integradas con la API de Mercado Libre, donde resolví entrega duplicada de webhooks, eventos desordenados, rate limiting y concurrencia entre usuarios.

En paralelo desarrollo en Java 21 y Spring Boot el backend de MiNutria, una app iOS publicada en App Store, con foco en autenticación, seguridad y testing. Uso IA como herramienta de desarrollo, manteniendo el criterio propio en el modelado de datos y las decisiones de arquitectura.

Habilidades técnicas

- **Backend:** Java 21, Spring Boot, Spring Security, Node.js, Express.js, APIs REST, arquitectura por capas
- **Bases de datos:** MySQL y PostgreSQL en producción, MongoDB en proyecto propio. Modelado de datos, índices, transacciones y control de concurrencia
- **Integraciones y sistemas distribuidos:** webhooks, colas de jobs, idempotencia, outbox pattern, retry con backoff y jitter, rate limiting, reconciliación, OAuth2, automatización con n8n
- **Seguridad:** JWT (emisión, validación y revocación), BCrypt, sesiones con cookies firmadas, control de acceso por rol con RLS en PostgreSQL
- **Testing y CI/CD:** JUnit 5, tests de servicios con mocks, smoke tests contra la API, GitHub Actions
- **Infraestructura:** Docker (build multi-stage), Docker Compose, Render, Redis, Socket.IO, SSE
- **Frontend y mobile:** HTML, CSS y JavaScript en producción; React, React Native, TypeScript y React Query en proyectos propios
- **Herramientas:** Git, GitHub, Maven

Experiencia profesional

Full Stack Developer — TodoMicro

Septiembre 2025 – Actualidad | Buenos Aires, Argentina

Responsable del desarrollo y la operación en producción de las aplicaciones internas con las que trabaja la empresa, con ownership técnico desde el diseño hasta el monitoreo.

- **Automatizador de precios y stock** entre Google Sheets, MySQL y la API de Mercado Libre para múltiples cuentas: endpoint de webhooks que persiste y encola sin procesar, deduplicación por clave de idempotencia con índice único, outbox pattern con latest-wins para eventos desordenados, retry con backoff y jitter, rate limiting por cuenta y job de reconciliación contra el estado real en Mercado Libre.
- **Freno de venta** (iniciativa propia, sin requerimiento previo): tras detectar cambios de precio hechos desde cuentas de venta comprometidas, diseñé un control en el callback de ventas que frena el procesamiento ante desvíos de precio, pausa las publicaciones del producto y deriva la venta a revisión manual, priorizando seguridad sobre disponibilidad.
- **Diagnóstico de falsos positivos** en esa validación: identifiqué que comparaba contra un precio mutable y no contra el vigente al momento de la venta. Rediseñé la regla para reconstruirlo desde la auditoría de cambios y la simulé contra el histórico antes de desplegarla: de 35 alertas, solo 4 eran reales.
- **Control de concurrencia** entre operadores en tres aplicaciones: la condición va dentro del UPDATE en lugar de leer y después escribir, y la reserva de preguntas usa lock pesimista con TTL en MySQL, sin depender de Redis.
- **Plataforma de logística en tiempo real** para revisión, cierre, predespacho y despacho de pedidos con pistola o cámara: transacciones con FOR UPDATE, reintento acotado ante deadlocks, outbox dentro de la transacción de despacho y cierre de pedido idempotente, optimizado de unos 24 viajes a la base a 3. Socket.IO y WebSocket para tiempo real, sesiones con cookies firmadas y JWT.
- **Generador de publicaciones** para crear publicaciones de Mercado Libre en varias cuentas a la vez y editarlas en masa: el alta no se reintenta porque no es idempotente (ante un 5xx busca por SKU si se creó), valida el GTIN antes de publicar y registra el resultado por cuenta para manejar errores parciales.

- **Bandeja centralizada de preguntas** de Mercado Libre para múltiples cuentas, con atribución de ventas por respuesta, y **panel de alertas** que detecta inconsistencias entre base, planilla y Mercado Libre.
- **Automatización del circuito de soporte con n8n** (self-hosted en Docker): los pedidos que los operadores crean en Google Tasks se convierten en tickets de Plane y se completan al cerrarse el ticket, con deduplicación y un workflow de errores. Lo llevé a producción en el servidor de la empresa, sin exponerlo a internet.

Tecnologías: Node.js, Express.js, MySQL, Redis, Socket.IO, n8n, Docker, HTML, CSS, JavaScript, Git.
Arquitectura: github.com/franpipito/automatizador-precios-stock

Gerente General — Pizzería La Nona

Enero 2021 – Septiembre 2025 | Lanús, Buenos Aires

- Coordiné las operaciones diarias y lideré un equipo de 5 personas.
- Implementé controles de inventario que redujeron los faltantes de insumos un 15% y digitalicé registros y seguimiento en Excel.

Proyectos

MiNutria — App de gastos personales y compartidos

Proyecto propio | App iOS publicada en App Store, con backend propio en producción

Stack: Java 21, Spring Boot, Spring Security, MongoDB, Maven, Docker, GitHub Actions, Render; cliente en React Native (Expo) y TypeScript.

- Filtro JWT escrito a mano sobre Spring Security, con revocación de tokens mediante un contador de versión que se valida contra la base en cada request, y login de tiempo constante (hash señuelo) para no exponer qué cuentas están registradas.
- Rate limiting por cuenta en login, recuperación de contraseña e invitaciones; recuperación con código hasheado, vencimiento a 15 minutos e intentos limitados; política de contraseñas según NIST SP 800-63B.
- Reglas críticas garantizadas por índices de MongoDB: guardado sin conexión y sin duplicados con clave de idempotencia e índice único parcial, códigos de invitación de un solo uso consumidos atómicamente con findAndModify, y un solo pozo común abierto por grupo.
- Validación de configuración antes de crear los beans, que impide arrancar en producción con secretos de desarrollo o una conexión a la base mal formada.
- 181 tests con JUnit 5 sobre lógica de dominio y servicios, CI con GitHub Actions en cada push y smoke test propio contra la API corriendo.

MatchPoint y MatchPoint Clubes — Co-fundador y desarrollador

Co-fundador, desarrollado entre dos | App publicada en App Store y panel web para clubes en producción

Plataforma para jugadores y clubes de pádel: partidos, resultados, ranking, chat, alquiler de canchas con seña y devoluciones. **Stack:** React Native (Expo), React, TypeScript, Supabase (PostgreSQL, RLS, RPC, Realtime, Edge Functions), React Query, MercadoPago.

- Pipeline de devoluciones de señas con MercadoPago: cola de reembolsos con índices únicos, cancelación por RPC que encola la devolución y Edge Function por cron con estados, hasta 5 intentos y clave de idempotencia por reserva.
- Cobro de señas con split de comisión y OAuth de MercadoPago por club, confirmado por webhook con firma HMAC e idempotencia sobre el id de pago.
- Resultados con estados (pendiente, confirmado, disputado, anulado) resueltos en RPCs de Postgres, ranking global y por club, y cancelación automática por cron de los partidos sin resultado confirmado a las 48 horas.
- Panel del club con React Query y reservas en tiempo real por Supabase Realtime filtrado por club; acceso por rol con RLS y acceso del jugador solo a través de RPCs controladas. Unos 470 casos de test con Jest y PGlite.

Arquitectura: github.com/franpipito/matchpoint-arquitectura

Educación

Ingeniería Informática — Universidad Argentina de la Empresa (UADE)

2025 – Actualidad (en curso)

Ingeniería Informática — Universidad Nacional Arturo Jauretche

2023 – 2025 | Promedio: 8,1

Idiomas

Español (nativo) · Inglés (intermedio, B1)